Bachelor of Computer Applications (Hons. Game Development)
Semester- III

Subject: Database Management System

Assignment 9: Advanced SQL Queries Using Joins and Subqueries

By
Name: FIDHA
Reg. No.: 2511021240021

Faculty Name: Renita Joy
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
October 2026

1. Introduction

Advanced reports combine related tables with JOIN operations and use subqueries where a derived set of records is needed. Each report below is aligned to the Library Management System schema finalized in Assignments 1 and 2.

7. Loan report with member, title, copy, and staff (JOIN)

Combines five related tables into one readable circulation report.

```
SELECT l.loan_id, m.name AS member, b.title, bc.copy_id,
 s.name AS issued_by, l.issue_date, l.due_date, l.return_date
FROM Loans l
JOIN Members m ON m.member_id = l.member_id
JOIN BookCopies bc ON bc.copy_id = l.copy_id
JOIN Books b ON b.book_id = bc.book_id
JOIN Staff s ON s.staff_id = l.staff_id
ORDER BY l.loan_id;
```

```
SELECT l.loan_id, m.name AS member, b.title, bc.copy_id,
       s.name AS issued_by, l.issue_date, l.due_date, l.return_date
FROM Loans l
JOIN Members m ON m.member_id = l.member_id
JOIN BookCopies bc ON bc.copy_id = l.copy_id
JOIN Books b ON b.book_id = bc.book_id
JOIN Staff s ON s.staff_id = l.staff_id
ORDER BY l.loan_id;
```

RESULT GRID (illustrative output from the sample dataset)

loan_id	member	title	copy_id	issued_by	issue_date	due_date	return_date
1001	Aarav Sharma	1984	101	Renuka Menon	2026-10-01	2026-10-15	2026-10-14
1002	Diya Nair	Harry Potter and the	103	Arjun Pillai	2026-10-03	2026-10-17	NULL
1003	Meera Iyer	Malgudi Days	106	Renuka Menon	2026-09-20	2026-10-04	2026-10-08
1004	Aarav Sharma	Sapiens	111	Sana Thomas	2026-10-05	2026-10-19	NULL
1005	Kabir Rao	A Brief History of Ti	109	Arjun Pillai	2026-09-01	2026-09-15	2026-09-20
1006	Ananya Sen	1984	102	Renuka Menon	2026-10-10	2026-10-24	NULL

8. Members with two or more loans (subquery)

Uses a subquery to find members who have borrowed at least twice.

```
SELECT m.member_id, m.name
FROM Members m
WHERE m.member_id IN (
 SELECT member_id
 FROM Loans
 GROUP BY member_id
 HAVING COUNT(*) &gt;= 2
 );
```

```
SELECT m.member_id, m.name
FROM Members m
WHERE m.member_id IN (
  SELECT member_id
  FROM Loans
  GROUP BY member_id
  HAVING COUNT(*) >= 2
);
```

RESULT GRID (illustrative output from the sample dataset)

member_id	name
1	Aarav Sharma

9. Fine summary by payment status

Summarizes number and value of paid versus unpaid fines.

SELECT paid_status, COUNT(*) AS fine_count,
SUM(amount) AS total_amount, ROUND(AVG(amount),2) AS average_amount
FROM Fines
GROUP BY paid_status;

```
SELECT paid_status, COUNT(*) AS fine_count,
       SUM(amount) AS total_amount, ROUND(AVG(amount),2) AS average_amount
FROM Fines
GROUP BY paid_status;
```

RESULT GRID (illustrative output from the sample dataset)

paid_status	fine_count	total_amount	average_amount
Paid	1	100.00	100.00
Unpaid	1	250.00	250.00

10. Overdue loans not yet returned (as of 2026-10-23)

Finds loans still open after their due date, using the assignment's sample date.

SELECT l.loan_id, m.name, b.title, l.due_date
FROM Loans l
JOIN Members m ON m.member_id = l.member_id
JOIN BookCopies bc ON bc.copy_id = l.copy_id
JOIN Books b ON b.book_id = bc.book_id
WHERE l.return_date IS NULL
AND l.due_date < '2026-10-23'
ORDER BY l.due_date;

```
SELECT l.loan_id, m.name, b.title, l.due_date
FROM Loans l
JOIN Members m ON m.member_id = l.member_id
JOIN BookCopies bc ON bc.copy_id = l.copy_id
JOIN Books b ON b.book_id = bc.book_id
WHERE l.return_date IS NULL
  AND l.due_date < '2026-10-23'
ORDER BY l.due_date;
```

RESULT GRID (illustrative output from the sample dataset)

loan_id	name	title	due_date
1002	Diya Nair	Harry Potter and the Philosopher's Ston	2026-10-17

11. Reservations with member and title

Displays each reservation together with the member and reserved title.

```
SELECT r.reservation_id, m.name, b.title, r.reservation_date, r.status
FROM Reservations r
JOIN Members m ON m.member_id = r.member_id
JOIN Books b ON b.book_id = r.book_id
ORDER BY r.reservation_id;
```

```
SELECT r.reservation_id, m.name, b.title, r.reservation_date, r.status
FROM Reservations r
JOIN Members m ON m.member_id = r.member_id
JOIN Books b ON b.book_id = r.book_id
ORDER BY r.reservation_id;
```

RESULT GRID (illustrative output from the sample dataset)

reservation_id	name	title	reservation_date	status
2001	Meera Iyer	Harry Potter and the Philosophe	2026-10-12	Pending
2002	Kabir Rao	1984	2026-10-13	Pending
2003	Ishaan Das	Sapiens	2026-10-14	Cancelled
2004	Diya Nair	A Brief History of Time	2026-10-15	Pending
2005	Ananya Sen	Wings of Fire	2026-10-16	Fulfilled

12. Book-author listing

Demonstrates the many-to-many relationship resolved by the junction table.

```
SELECT b.title, a.name AS author
FROM Book_Author ba
JOIN Books b ON b.book_id = ba.book_id
JOIN Authors a ON a.author_id = ba.author_id
ORDER BY b.title;
```

```
SELECT b.title, a.name AS author
FROM Book_Author ba
JOIN Books b ON b.book_id = ba.book_id
JOIN Authors a ON a.author_id = ba.author_id
ORDER BY b.title;
```

RESULT GRID (illustrative output from the sample dataset)

title	author
1984	George Orwell
A Brief History of Time	Stephen Hawking
Animal Farm	George Orwell
Harry Potter and the Philosopher's Stone	J.K. Rowling
Malgudi Days	R.K. Narayan
Pride and Prejudice	Jane Austen
Sapiens	Yuval Noah Harari
Wings of Fire	A.P.J. Abdul Kalam

2. Additional report: count loans per member

```
SELECT m.member_id, m.name, COUNT(l.loan_id) AS total_loans
FROM Members m
LEFT JOIN Loans l ON l.member_id = m.member_id
GROUP BY m.member_id, m.name
ORDER BY total_loans DESC, m.name;
```

This report includes members with zero loans because it uses LEFT JOIN. It can help the library understand circulation activity by member.

3. Conclusion

The reports demonstrate multi-table joins, grouping, aggregate functions, and a subquery. They can support practical tasks such as identifying overdue loans, reviewing reservations, checking fine balances, and understanding borrowing activity.